

Training Dataset Annotation

Training Dataset Annotation

1. Course Content
2. Example Dataset
3. Traffic Sign Dataset Annotation
 - 3.1 Label Studio Introduction
 - 3.2 Start Label Studio
 - 3.3 Access Label Studio
 - 3.4 Create Project
 - 3.5 Import Images
 - 3.6 Label Settings
 - 3.7 Dataset Annotation
 - 3.8 Export Dataset
 - 3.9 Organize Dataset Directory Structure
 - 3.10 Dataset Configuration File
4. Lane Detection Dataset Annotation
 - 4.1 Create Project
 - 4.2 Organize Exported Dataset
 - 4.3 Dataset Configuration File

[!IMPORTANT]

- The factory image already includes preset datasets required for training traffic sign recognition and lane detection models. This tutorial is provided for users who need to train their own YOLOv11 models and want to learn how to collect and annotate datasets
- If you want to experience the complete functionality directly, you can skip this section!!!
- This tutorial is mainly conducted on the Orin Nano board

1. Course Content

1. Master the data annotation methods required for training YOLOv11 object detection model datasets

2. Example Dataset

- **Only Orin boards are recommended for on-board model training. RDK boards have insufficient performance and require annotation and training on a host with GPU. The training source code is in the yolo_train folder under Model Training in the source code summary. If training is needed, you can place the yolo_train folder programs in the root directory of the board**
- Traffic sign dataset path

```
/home/jetson/yolo_train/train_sign/roadsign_dataset.zip
```

- Lane dataset path

```
/home/jetson/yolo_train/train_lane/lane_dataset.zip
```

3. Traffic Sign Dataset Annotation

3.1 Label Studio Introduction

Label Studio is an open-source data annotation platform for data annotation and annotation task management, supporting various types of data input and annotation formats.

The screenshot shows the Label Studio website. At the top, it says "Open Source Data Labeling Platform" with "Data Labeling" in a red box. Below this, it states: "The most flexible data labeling platform to fine-tune LLMs, prepare training data, or evaluate AI models." There are two buttons: "Download OSS" and "Compare Versions". Below the buttons, it says "LAST COMMIT: NOVEMBER 17, 2025 | LATEST VERSION: 1.21.0". On the right, there is a "Quick Start" section with a terminal window showing the following commands:

```
1 # Install the package
2 # into python virtual environment
3 pip install -U label-studio
4 # Launch it!
5 label-studio
```

Below the terminal window is a cartoon illustration of a dog. At the bottom, it says "Label every data type." with a row of buttons: "GENAI", "IMAGES", "AUDIO", "TEXT", "TIME SERIES", "MULTI-DOMAIN", and "VIDEO".

[!NOTE]

The factory image environment already has Label Studio installed!!! If users need to set up a data annotation environment on their own computers, they need to install Label Studio first

```
pip install label-studio
```

3.2 Start Label Studio

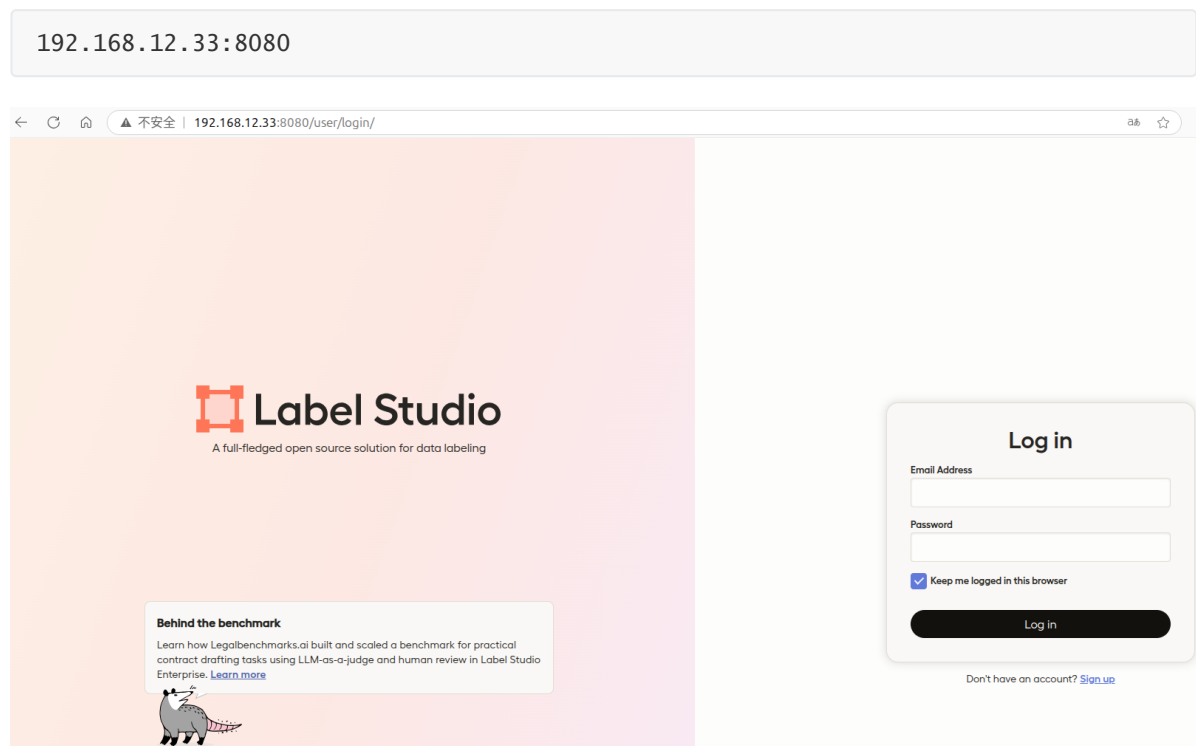
- Start Label Studio

```
sudo docker run -it -p 8080:8080 -v
/home/jetson/ultralytics/ultralytics/data:/label-studio/data heartexlabs/label-
studio:latest label-studio
```

```
jetson@yahboom: ~  
jetson@yahboom:~$ docker run -it -p 8080:8080 -v /home/jetson/ultralalytics/ultralalytics/data:/label-studio/data heartexlabs/label-studio:latest label-studio --log-level DEBUG  
=> Database and media directory: /label-studio/data  
=> Static URL is set to: /static/  
=> Database and media directory: /label-studio/data  
=> Static URL is set to: /static/  
Read environment variables from: /label-studio/data/.env  
get 'SECRET_KEY' casted as '<class 'str'>' with default ''  
/label-studio/.venv/lib/python3.12/site-packages/label_studio_sdk/_extensions/label_studio_tools/core/label_config.py:137: SyntaxWarning: invalid escape sequence '\$'  
    expression = "^\\$[A-Za-z_]+$"  
Starting new HTTPS connection (1): pypi.org:443  
https://pypi.org:443 "GET /pypi/label-studio/json HTTP/1.1" 200 33651  
[2024-12-31 09:12:15,565] [core.feature_flags.base::<module>::40] [INFO] Read flags from file /label-studio/label_studio/feature_flags.json  
[2024-12-31 09:12:16,078] [core.redis::<module>::122] [DEBUG] => Redis is not connected.  
[2024-12-31 09:12:16,502] [faker.factory::<module>::20] [DEBUG] Not in REPL -> leaving logger event level as is.  
[2024-12-31 09:12:18,045] [faker.factory::_find_provider_class::78] [DEBUG] Looking for locale 'en_US' in provider 'faker.providers.address'.  
[2024-12-31 09:12:18,048] [faker.factory::_find_provider_class::97] [DEBUG] Prov
```

3.3 Access Label Studio

Make sure the computer and the vehicle are in the same local network. Enter the vehicle's IP + port 8080 in the browser address bar. The vehicle's IP can be viewed from the OLED screen in front of the vehicle or by opening a new terminal. Here we use `IP:192.168.12.33` as an example:



- Default account information (all locally stored information, no data leakage risk):

Account: yahboom@163.com
Password: yahboom@163.com

- If you need to create a new account, click `sign up`



Label Studio

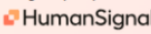
A full-fledged open source solution for data labeling

Did you know?

Label Studio now has a Starter Cloud offering optimized for small teams and projects. [Learn more](#)



Brought to you by



Log in

Email Address

Password

☒ Keep me logged in this browser

Log in

Don't have an account? [Sign up](#)



Label Studio

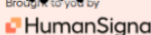
A full-fledged open source solution for data labeling

Did you know?

Label Studio now has a Starter Cloud offering optimized for small teams and projects. [Learn more](#)



Brought to you by



Sign Up

Email Address

Password

How did you hear about Label Studio?

Github

☐ Get the latest news from Heidi 

Create Account

Already have an account? [Log in](#)

After registration is completed, the website will automatically log in:

Label Studio



Heidi doesn't see any projects here!

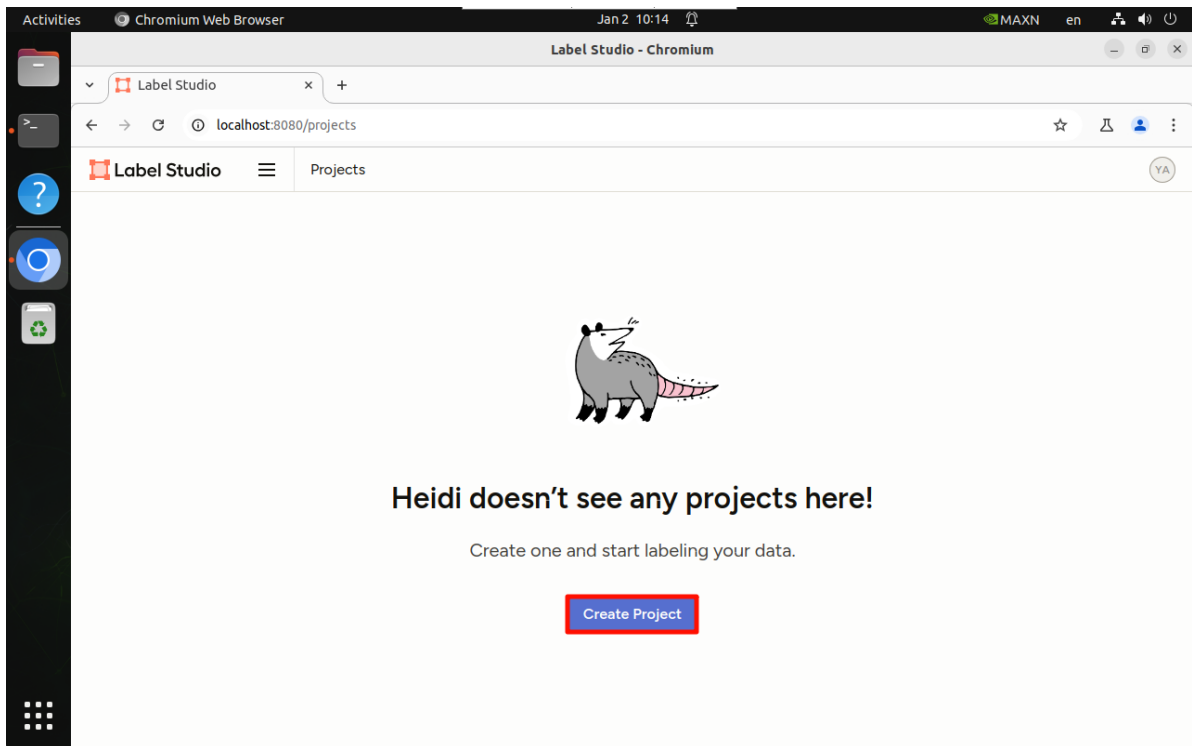
Create one and start labeling your data.

Create Project

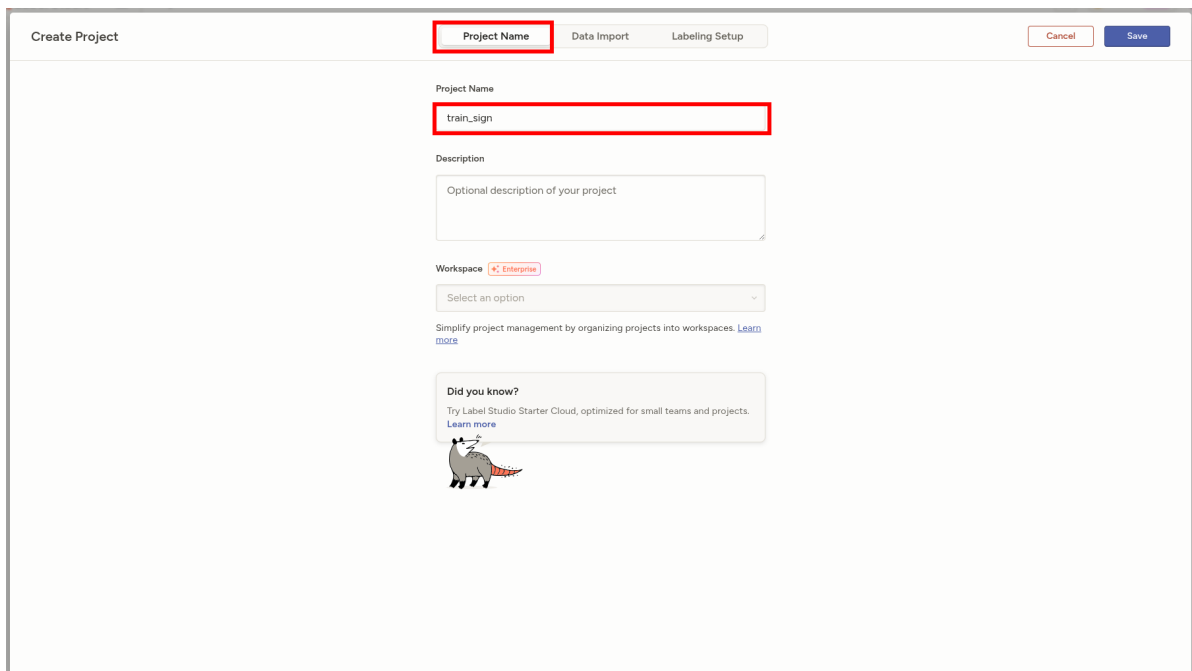
Account & Settings

Log Out

3.4 Create Project

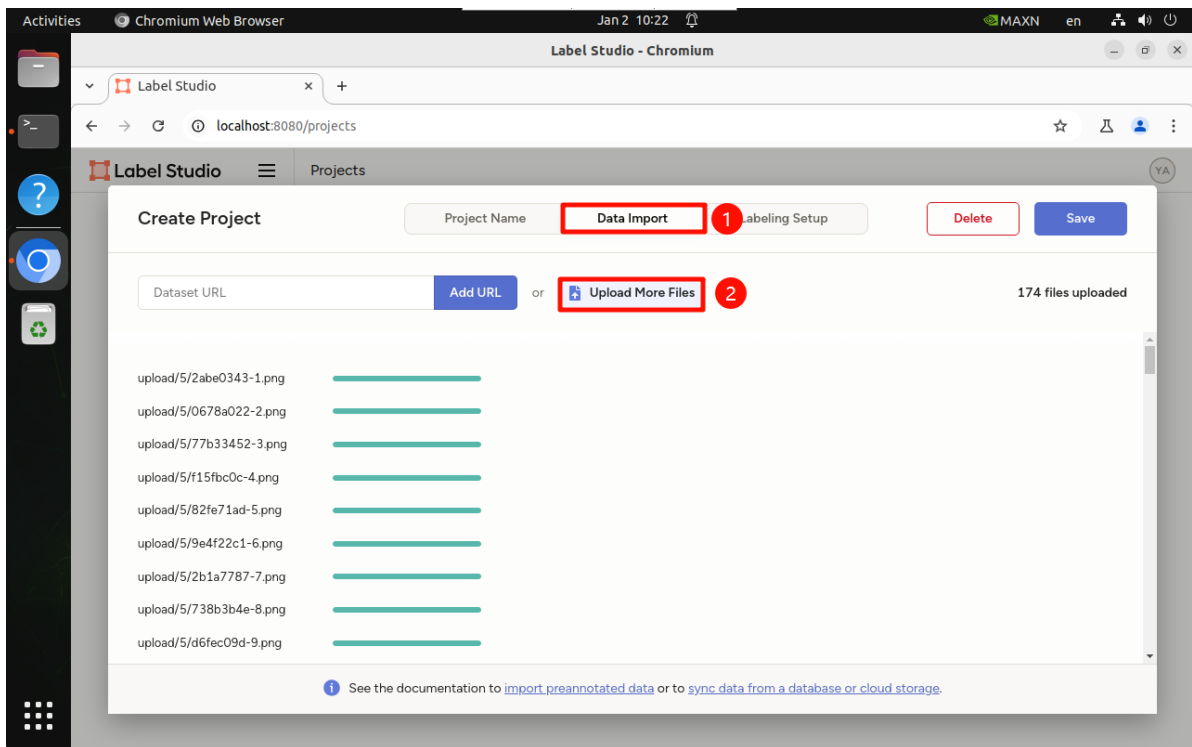


The project name can be named arbitrarily, name it according to your training set:



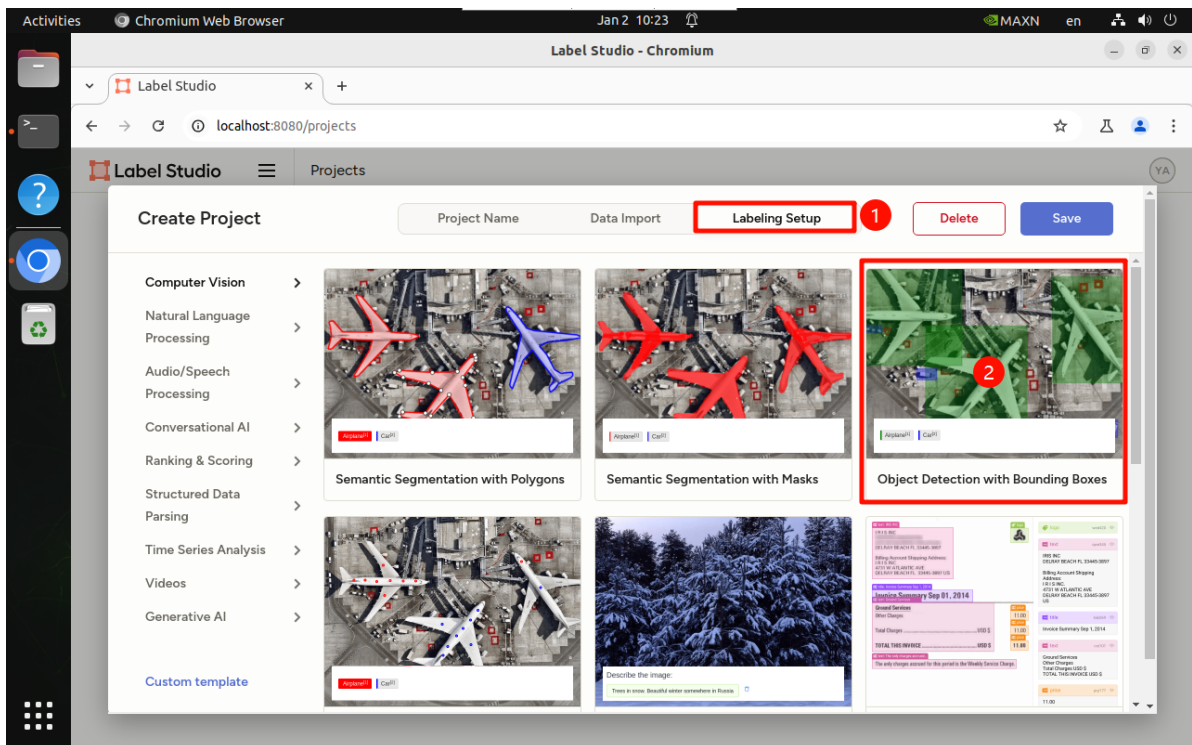
3.5 Import Images

- Import the images previously split from the video: cannot import too many images at once, it is recommended to select 50-100 images each time, then import multiple times

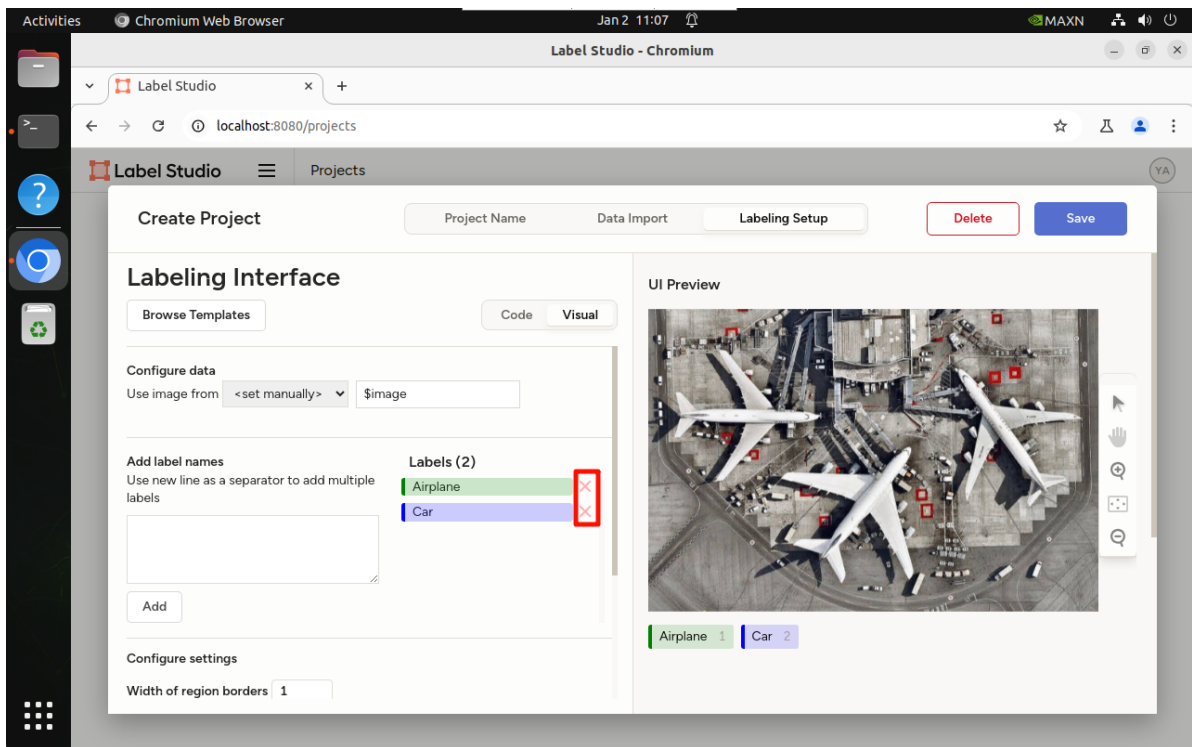


3.6 Label Settings

We are demonstrating orange recognition, so select "Object Detection"



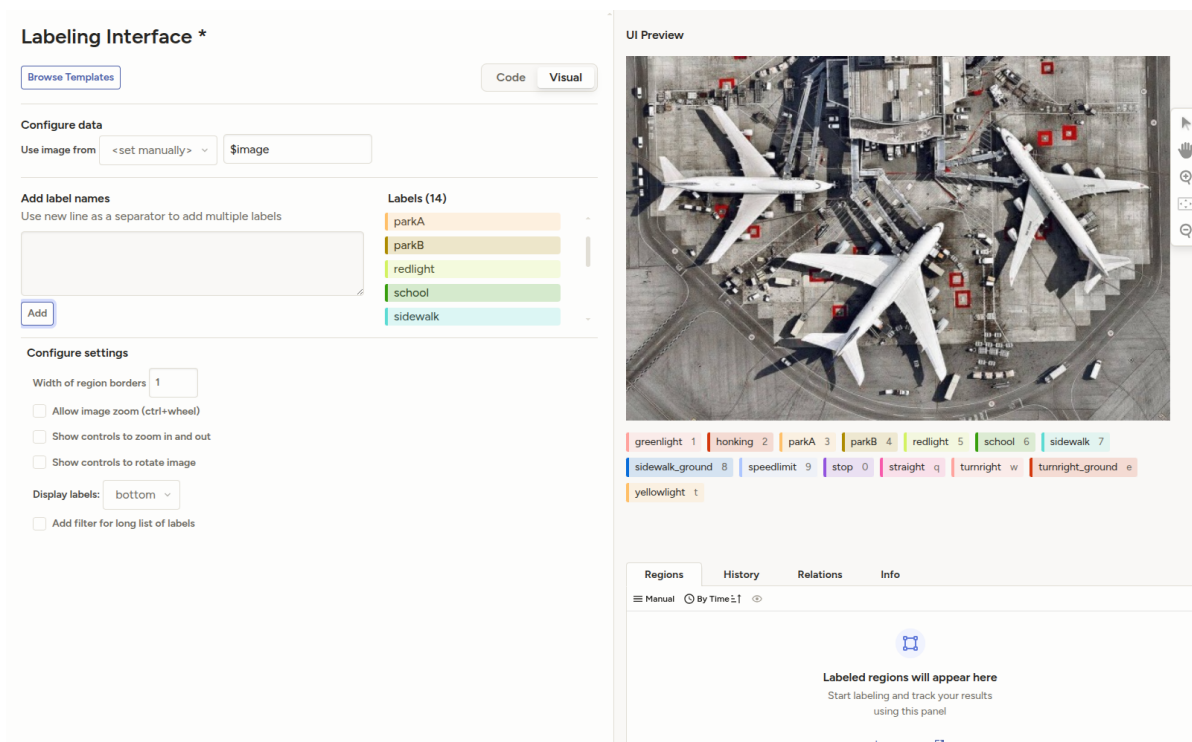
- Delete built-in labels:



- Add new labels, here in the example we name all traffic signs and ground markings in the sandbox map

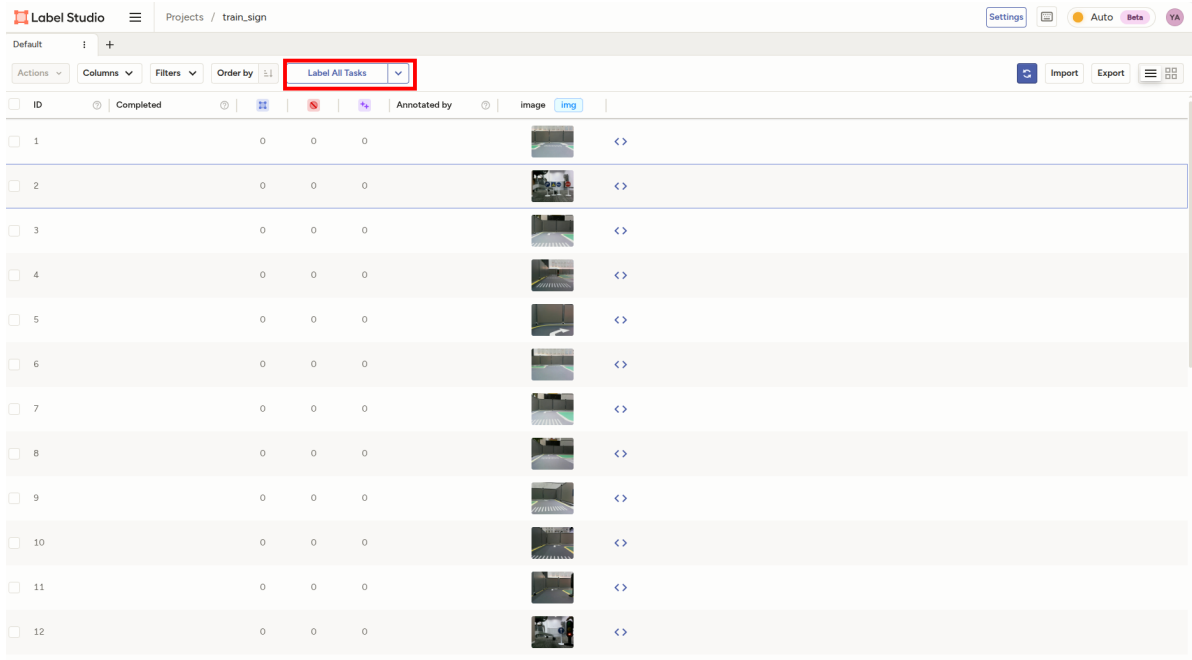
[!TIP]

If users want to train other types of object detection models, and the detection targets are not traffic signs or lanes, name the labels according to the targets that need to be detected in your dataset.

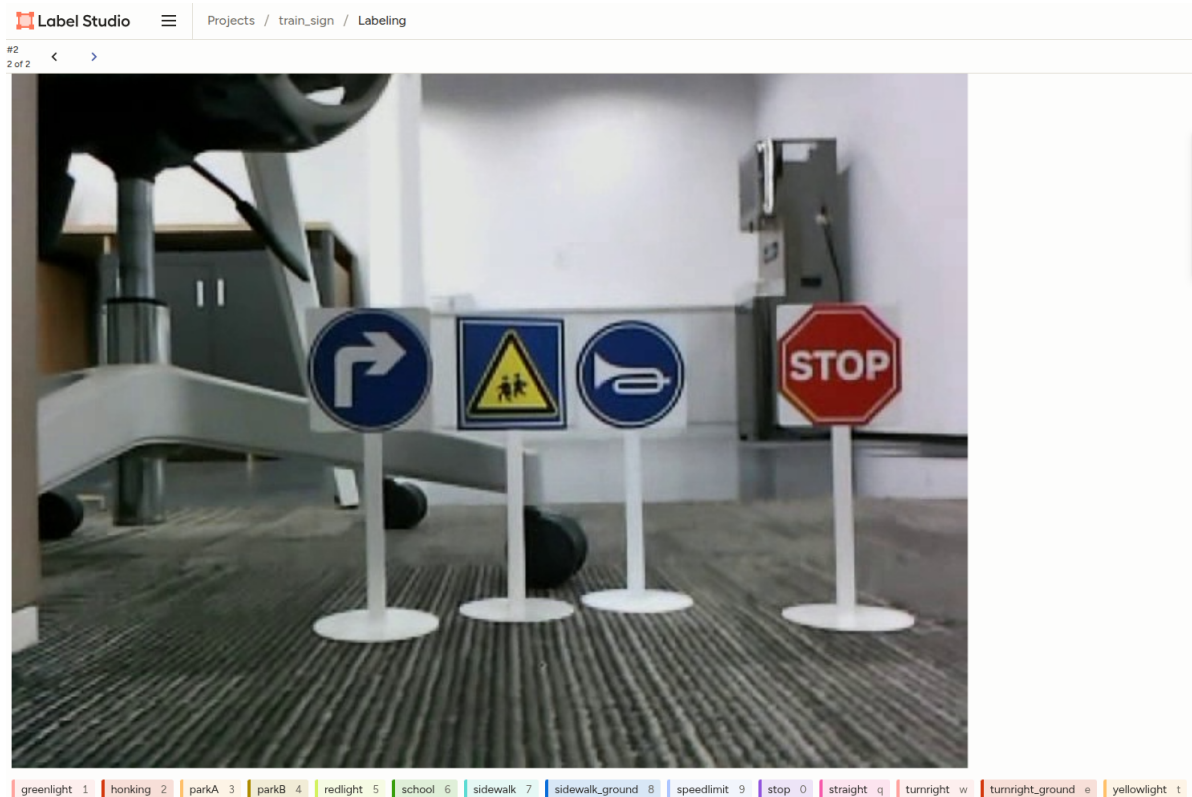


3.7 Dataset Annotation

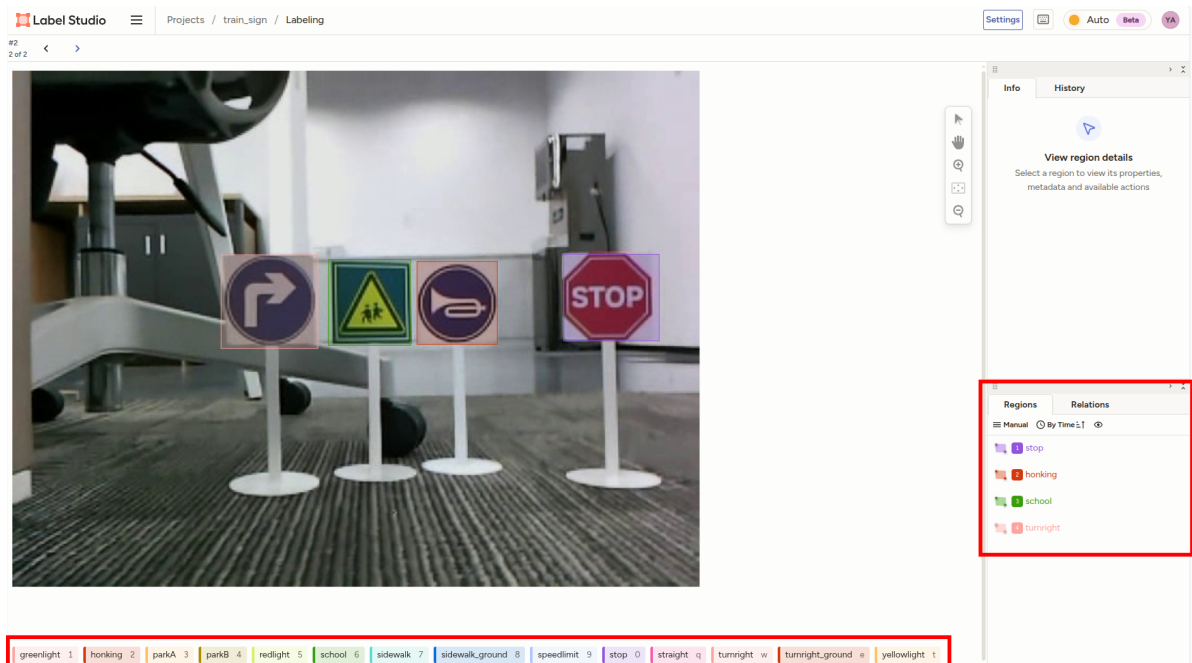
- Click Annotate All Tasks:



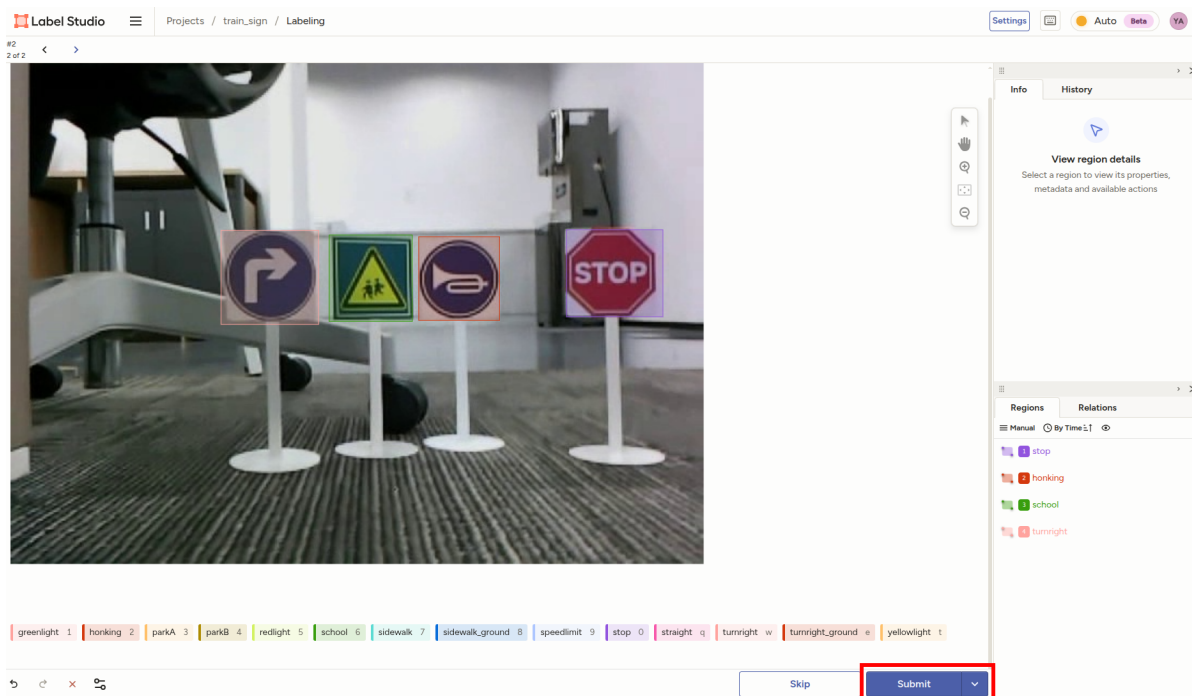
- The corresponding target category shortcuts are displayed on the labels below. Press the corresponding shortcut key on the keyboard to quickly select that label.



- Then drag the mouse to select the corresponding target. There are 4 types of targets in this image. After marking is completed, the marked labels will be displayed in the **Regions** column on the right, where you can check whether the marking is correct.



- After marking is completed, click **submit** to submit, and it will automatically jump to the next image. Repeat annotation until all image data annotation is completed.



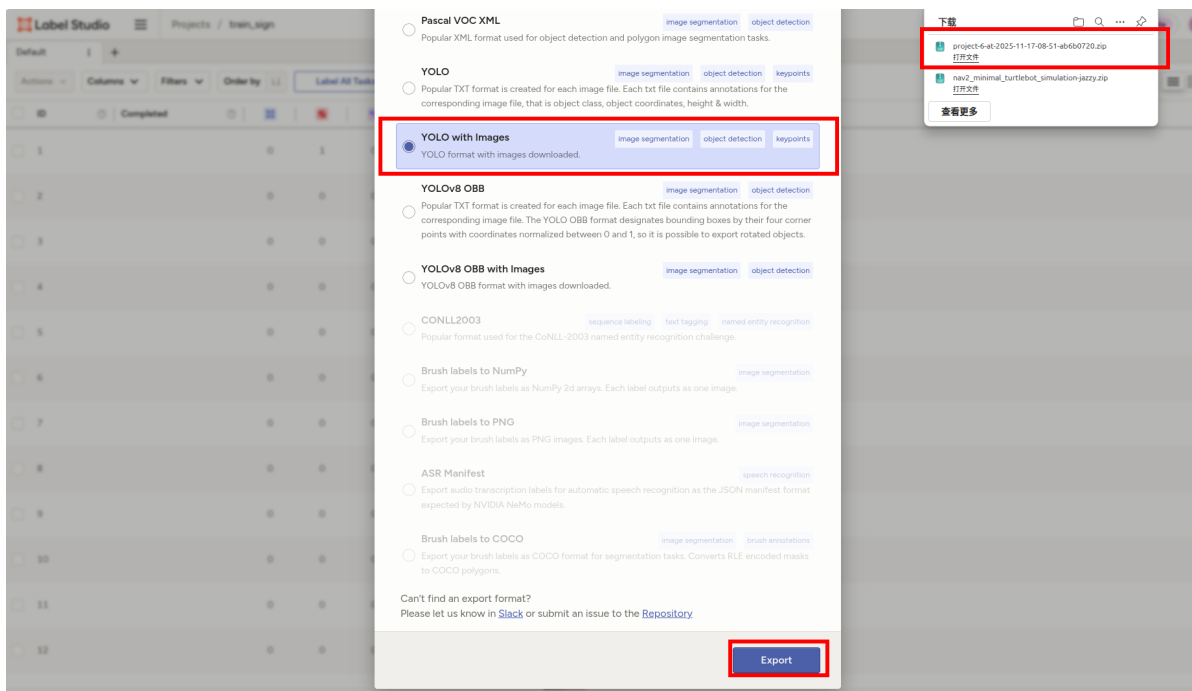
3.8 Export Dataset

- Select the project to export, click **Export** in the upper right corner

ID	Completed			Annotated by	image	
1	0	1	0	YA		<>
2	0	0	0			<>
3	0	0	0			<>
4	0	0	0			<>
5	0	0	0			<>
6	0	0	0			<>
7	0	0	0			<>
8	0	0	0			<>
9	0	0	0			<>
10	0	0	0			<>
11	0	0	0			<>
12	0	0	0			<>

Tasks: 62 / 62 Submitted annotations: 0 Predictions: 0

- Select **YOLO with Images** format for export, the browser will automatically download a compressed package



3.9 Organize Dataset Directory Structure

- The directory structure corresponding to the compressed package exported using Label Studio is as follows:

```

.
├── images
├── labels
├── classes.txt
└── notes.json

```

Folder and file description:

- images: stores original images

- labels: annotation data corresponding to each image
- classes.txt: target classification
- notes.json: target classification and corresponding numbers

- To enable YOLOv11 to recognize the dataset, some adjustments need to be made to the directory structure of data exported from Label Studio
- Divide the images and labels exported from Label Studio into train (training set) and val (validation set). The images and labels in both train and val need to have corresponding names
- Here for the example, directly copy the images and labels under the train directory into the val directory

```
.
├── classes.txt
├── notes.json
├── train
│   ├── images
│   └── labels
└── val
    ├── images
    └── labels
```

- The following is the directory structure of the tutorial example dataset (the factory vehicle image and the dataset provided in this tutorial folder are in .zip compressed package format, need to be decompressed first to see the complete directory structure)

A terminal window screenshot showing a directory tree command. The terminal title is 'jetson@yahboom: ~/yahboomM3_ws/yolo_train/train_sign/roadsign_dataset'. The command entered is 'tree -L 2'. The output shows a directory structure with 'classes.txt', 'notes.json', 'train' (containing 'images', 'labels', 'labels.cache'), and 'val' (containing 'images', 'labels', 'labels.cache'). At the bottom, it says '6 directories, 4 files'.

```
jetson@yahboom: ~/yahboomM3_ws/yolo_train/train_sign/roadsign_dataset
jetson@yahboom:~/yahboomM3_ws/yolo_train/train_sign/roadsign_dataset$ tree -L 2
.
├── classes.txt
├── notes.json
├── train
│   ├── images
│   ├── labels
│   └── labels.cache
└── val
    ├── images
    ├── labels
    └── labels.cache
6 directories, 4 files
```

3.10 Dataset Configuration File

- When training YOLOv11 models, you need to specify the dataset path and detection target categories through a YAML format configuration file
- Traffic sign dataset configuration file path:

```
/home/jetson/yolo_train/train_sign/sign.yaml
```

Content analysis:

- path: dataset path
- train: training set path
- val: validation set path

- names: detection target categories (need to be consistent with the serial number categories in notes.json exported from Label Studio)

```
path: /home/jetson/yolo_train/roadsign_dataset # dataset root dir
train: train/images
val: val/images

# Classes
names:
  0: greenlight
  1: honking
  2: parkA
  3: parkB
  4: redlight
  5: school
  6: sidewalk
  7: sidewalk_ground
  8: speedlimit
  9: stop
  10: straight
  11: turnright
  12: turnright_ground
  13: yellowlight
```

4. Lane Detection Dataset Annotation

Lane detection dataset annotation is basically the same as traffic sign dataset annotation process, both use YOLOv11 object detection model for object detection (here we recommend using object detection model, do not recommend using segmentation model for lane detection, model inference speed is slower)

4.1 Create Project

- The creation process is the same as before, here fill `train_lane` in `Project Name`

Create Project

Project Name: train_lane

Description: Optional description of your project

Workspace: Enterprise

New Storage Connector

You can connect Label Studio Enterprise to Databricks Unity Catalog (UC) Volumes to import files as tasks and export annotations. [Learn more](#)

- Here the lane detection model category has only one, fill in `lane`

Create Project

Project NameData ImportLabeling Setup

CancelSave

Labeling Interface

Browse TemplatesCodeVisual

Configure data

Use image from<set manually>\$image

Add label names

Use new line as a separator to add multiple labels

lane

Add

Labels (1)

lane

Configure settings

Width of region borders1

☐ Allow image zoom (ctrl+wheel)
 ☐ Show controls to zoom in and out
 ☐ Show controls to rotate image

Display labels:bottom

☐ Add filter for long list of labels

UI Preview

- Import the split lane video frame dataset

Import Data

CancelImport

Dataset URL

Add URL

or

Upload More Files

11 files uploaded

Files

upload/7/f5f3e7d5-ebacce16-494.jpg	43.1 KB
upload/7/78f619e3-feca2652-552.jpg	52 KB
upload/7/a484b7d7-fe517684-97.jpg	44.6 KB
upload/7/c9386389-fca94b45-587.jpg	62.07 KB
upload/7/29b1e680-fc450179-10.jpg	47.16 KB
upload/7/8feca5a6-fc31baba-462.jpg	43.27 KB
upload/7/94b68863-fbae8f6b-15.jpg	55 KB
upload/7/47221973-fa645c82-31.jpg	50.09 KB
upload/7/231b5bf0-fa5a7b0-407.jpg	27.32 KB
upload/7/9ce4b882-fa3077a3-507.jpg	39.01 KB
upload/7/f67ef9e-f9bee1d8-569.jpg	59.97 KB

- The annotation method is the same as before, here we box and annotate all lane lines

Label Studio

Projects / train_lane / Labeling

SettingsAutoBetaV4

#70
8 of 8

InfoHistory

View region details

Select a region to view its properties, metadata and available actions

RegionsRelations

ManualBy Time: 1

lane

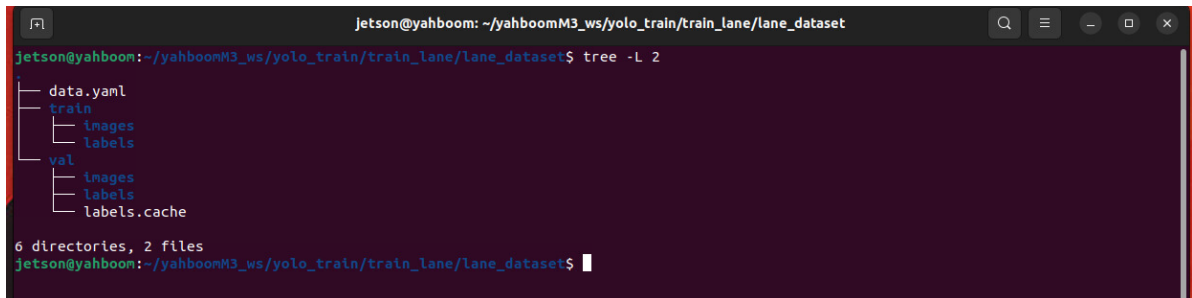
lane

lane

SkipSubmit

4.2 Organize Exported Dataset

- After annotation is completed, we export and organize the dataset into a target structure that YOLO can recognize. The method is completely consistent with the previous traffic sign dataset export and organization method
- Reference target structure is as follows:

A terminal window with a dark background. The title bar shows the user 'jetson@yahboom' and the path '~/yahboomM3_ws/yolo_train/train_lane/lane_dataset'. The command 'tree -L 2' has been executed, showing a tree structure: 'data.yaml' at the root, with children 'train' and 'val'. 'train' has children 'images' and 'labels'. 'val' has children 'images', 'labels', and 'labels.cache'. Below the tree, it says '6 directories, 2 files'. The prompt is 'jetson@yahboom:~/yahboomM3_ws/yolo_train/train_lane/lane_dataset\$' with a cursor.

```
jetson@yahboom: ~/yahboomM3_ws/yolo_train/train_lane/lane_dataset
jetson@yahboom:~/yahboomM3_ws/yolo_train/train_lane/lane_dataset$ tree -L 2
.
├── data.yaml
├── train
│   ├── images
│   └── labels
└── val
    ├── images
    ├── labels
    └── labels.cache

6 directories, 2 files
jetson@yahboom:~/yahboomM3_ws/yolo_train/train_lane/lane_dataset$
```

4.3 Dataset Configuration File

- Lane detection dataset configuration file path:

```
/home/jetson/yolo_train/train_lane/lane.yaml
```

- Example configuration file is as follows:

```
path: /home/jetson/yolo_train/train_lane/lane_dataset # dataset root dir
train: train/images
val: val/images

# Classes
names:
  0: lane
```